

Day 5: NLP Fundamentals for IBM AI/GenAI Preparation

1. 5-line beginner summary

NLP means teaching computers to understand and work with human language. Text data is usually unstructured, so we clean and convert it into numbers before using ML models. Older NLP methods use techniques like Bag of Words and TF-IDF. Modern NLP uses embeddings, transformers, LLMs, and RAG systems. For IBM AI/GenAI roles, NLP is important because LLMs, chatbots, search, document Q&A, and RAG all depend on text understanding.

2. What NLP is

NLP = Natural Language Processing

It is a field of AI that helps computers understand, process, analyze, and generate human language.

Human language can be:

```
Emails
Chat messages
PDF documents
Customer reviews
Support tickets
Medical notes
Legal contracts
Resume text
News articles
```

A computer does not naturally understand meaning like humans. So NLP converts text into a form that machines can process.

Example:

```
User text:
"I am not happy with the internet speed."

NLP can detect:
- Topic: internet speed
- Sentiment: negative
- Intent: complaint
- Important entity: internet service
```

In real AI projects, NLP is used for:

```
Chatbots
Document Q&A
Sentiment analysis
Text classification
Resume screening
Email routing
Customer support automation
```

Search systems
RAG pipelines
LLM applications

3. Structured vs unstructured data

Structured data

Structured data is organized in rows and columns.

Example:

Customer ID	Name	Age	City
101	Rahul	32	Pune
102	Priya	29	Delhi

This type of data is easy for machines to read.

Usually stored in:

SQL tables
Excel files
CSV files
Databases

Unstructured data

Unstructured data does not have a fixed table format.

Example:

"The customer complained that the internet speed is very poor and the service provider is asking for extra charges."

This has useful information, but it is hidden inside free text.

Usually found in:

Emails
PDFs
Chat messages
Reviews
Tickets
Audio transcripts
Documents

Simple comparison

Point	Structured Data	Unstructured Data
Format	Rows and columns	Free text, documents, images, audio
Easy for ML?	Yes	Needs preprocessing
Example	Customer table	Customer complaint email
Common in NLP?	Less	Very common
Processing need	Cleaning, scaling	Text cleaning, tokenization, embeddings

4. Text preprocessing

Text preprocessing means cleaning raw text before using it in an NLP model.

Raw text usually contains noise.

Example raw text:

```
"Hello!!! I am soooo happy with IBM's AI product 😊 Visit https://example.com"
```

After preprocessing:

```
"hello happy ibm ai product"
```

Common preprocessing steps:

```
Lowercasing
Removing punctuation
Removing extra spaces
Removing URLs
Removing emojis if not useful
Removing stop words
Tokenization
Stemming or lemmatization
```

But be careful: preprocessing depends on the model.

For traditional ML, preprocessing is very important.

For modern LLMs, heavy preprocessing is usually not required because LLMs are trained to handle natural text.

5. Tokenization

Tokenization means breaking text into smaller pieces.

These pieces are called **tokens**.

Example:

Sentence:
"I love machine learning"

Word tokens:
["I", "love", "machine", "learning"]

Another example:

Sentence:
"IBM uses watsonx for AI."

Tokens:
["IBM", "uses", "watsonx", "for", "AI", "."]

In traditional NLP, tokens are usually words.

In LLMs, tokens may be:

Words
Subwords
Characters
Parts of words

Example:

"unbelievable"

May become:
["un", "believ", "able"]

Why tokenization matters:

Models do not understand full sentences directly.
They first break text into tokens.
Then tokens are converted into numbers.
Then the model processes those numbers.

6. Stop words

Stop words are very common words that may not add much meaning in some tasks.

Examples:

is
am
are
the
a
an
and
or
to
for

```
in
on
```

Example:

```
Original:
"The customer is not happy with the service"

After removing stop words:
"customer not happy service"
```

But be careful.

Sometimes stop words are important.

Example:

```
"I am happy"
"I am not happy"
```

The word **not** is very important.

If we remove not, the meaning changes completely.

So for sentiment analysis, do not blindly remove stop words.

7. Stemming and lemmatization

Both are used to reduce words to their base form.

Stemming

Stemming cuts words roughly.

Example:

```
playing  -> play
played   -> play
player   -> play
studies  -> studi
```

Stemming is fast but sometimes inaccurate.

Example:

```
"studies" -> "studi"
```

This is not a proper English word.

Lemmatization

Lemmatization converts words to their proper dictionary form.

Example:

playing	-> play
played	-> play
better	-> good
mice	-> mouse

Lemmatization is more accurate but slower.

Simple comparison

Point	Stemming	Lemmatization
Method	Cuts word endings	Uses grammar and dictionary
Speed	Faster	Slower
Accuracy	Lower	Higher
Output	May not be real word	Usually real word
Example	studies → studi	studies → study

8. Bag of Words

Bag of Words is an old but important NLP technique.

It converts text into numbers by counting word frequency.

Example documents:

Doc 1: "I love AI"
Doc 2: "I love NLP"
Doc 3: "AI and NLP are useful"

Vocabulary:

["I", "love", "AI", "NLP", "and", "are", "useful"]
--

Numerical representation:

Document	I	love	AI	NLP	and	are	useful
Doc 1	1	1	1	0	0	0	0
Doc 2	1	1	0	1	0	0	0
Doc 3	0	0	1	1	1	1	1

Bag of Words ignores word order.

Example:

"Dog bites man"
"Man bites dog"

Bag of Words may see them as very similar, even though meaning is different.

9. TF-IDF

TF-IDF means:

Term Frequency - Inverse Document Frequency

It gives importance to words.

Simple meaning:

A word is important if:

- It appears frequently in one document
- But does not appear in every document

Example:

Common word:

"the" appears everywhere, so it is less important.

Specific word:

"refund" appears in complaint emails about payment, so it is more useful.

TF-IDF is better than simple word count because it reduces the importance of very common words.

Example:

Document:

"I want refund for failed payment"

Important words:

refund
failed
payment

Less important words:

I
want
for

TF-IDF is useful for:

Search
Text classification
Document similarity
Keyword extraction
Traditional ML models

10. Word embeddings

Word embeddings convert words into dense numerical vectors.

A vector is just a list of numbers.

Example:

```
"king" -> [0.21, -0.45, 0.88, ...]
"queen" -> [0.25, -0.40, 0.91, ...]
"apple" -> [-0.76, 0.12, 0.33, ...]
```

The key idea:

Words with similar meaning have similar vectors.

Example:

```
king and queen are close
car and vehicle are close
doctor and hospital are close
```

Word embeddings capture meaning better than Bag of Words or TF-IDF.

Classic word embedding models:

```
Word2Vec
GloVe
FastText
```

Modern LLMs also use embeddings internally.

11. Sentence embeddings

Sentence embeddings convert a full sentence, paragraph, or document chunk into a vector.

Example:

```
Sentence:
"Customer wants refund for failed payment"

Embedding:
[0.12, -0.55, 0.77, 0.31, ...]
```

Sentence embeddings are useful because they capture overall meaning.

Example:

```
Sentence 1:
"I want my money back."

Sentence 2:
"Please process my refund."

These sentences use different words but have similar meaning.
Sentence embeddings can understand that they are related.
```

Used in:

```
Semantic search
RAG
```

Document Q&A
Duplicate question detection
Recommendation systems
Chatbot memory

12. Text classification

Text classification means assigning a category to text.

Example:

Input:
"My internet is not working."

Output:
Category = Technical Complaint

Other examples:

Email classification:
Spam / Not Spam

Review classification:
Positive / Negative / Neutral

Ticket classification:
Billing / Technical / Refund / Account

Document classification:
Invoice / Contract / Resume / Medical Report

Traditional text classification flow:

Text
→ Clean text
→ Convert text to numbers using TF-IDF
→ Train ML model
→ Predict class

Common models:

Logistic Regression
Naive Bayes
Random Forest
SVM
Transformer models
LLMs

13. Named Entity Recognition

Named Entity Recognition means finding important named things in text.

These are called entities.

Example:

Text:
"Ramraj Tiwari lives in Pune and works with IBM."

NER output:
Ramraj Tiwari -> Person
Pune -> Location
IBM -> Organization

Common entity types:

Person
Organization
Location
Date
Money
Product
Email
Phone number
Medical condition
Policy number

NER is useful in enterprise systems.

Example:

Insurance document:
"Patient Vidyawati Devi was diagnosed with hypothyroidism in 2018."

NER can extract:
Person: Vidyawati Devi
Condition: hypothyroidism
Year: 2018

NER is used for:

Document extraction
Invoice processing
Resume parsing
Medical record analysis
Legal document review
Knowledge graph creation

14. Sentiment analysis

Sentiment analysis means detecting emotion or opinion from text.

Example:

Text:
"I am very happy with the support."

Sentiment:
Positive

Example:

Text:
"The service is very poor and I am frustrated."

Sentiment:
Negative

Common sentiment labels:

Positive
Negative
Neutral
Mixed

Used in:

Customer reviews
Social media monitoring
Support ticket priority
Product feedback
Brand analysis
Call center analytics

Important point:

Sentiment analysis is not always easy.

Example:

"Great, the internet is down again."

This may look positive because of "Great",
but actually it is negative sarcasm.

Modern LLMs handle such cases better than simple keyword-based systems.

15. How NLP connects to LLMs and RAG

LLMs are advanced NLP models.

Traditional NLP usually performs specific tasks:

Classify this email
Extract names from this document
Find sentiment of this review
Search similar documents

LLMs can perform many language tasks using one model:

Summarize this document
Answer this question
Translate this text
Extract entities
Generate email response
Explain code
Create chatbot response

NLP to LLM connection

LLMs still depend on NLP basics:

Tokenization
Embeddings
Language modeling
Context understanding
Text generation
Semantic similarity

When you send a prompt to an LLM:

Prompt text
→ Tokenization
→ Embedding representation
→ Transformer processing
→ Generated answer

NLP to RAG connection

RAG means Retrieval-Augmented Generation.

It combines:

Search + LLM generation

RAG is used when an LLM needs to answer from private or enterprise documents.

Example:

User asks:
"What is the refund policy for cancelled booking?"

RAG system:

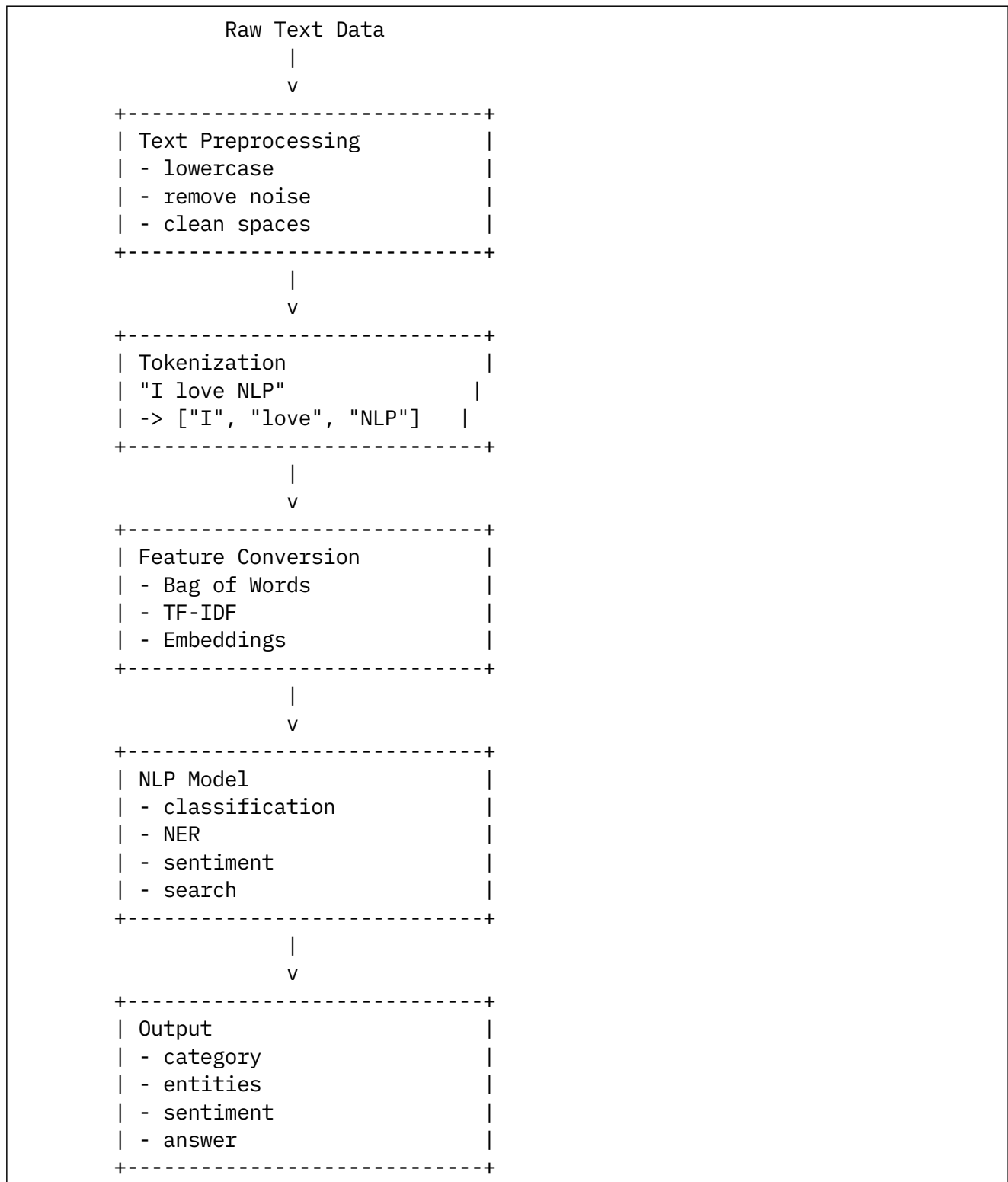
1. Searches company documents
2. Finds relevant refund policy
3. Sends policy text to LLM
4. LLM generates answer using that document

NLP is used in RAG for:

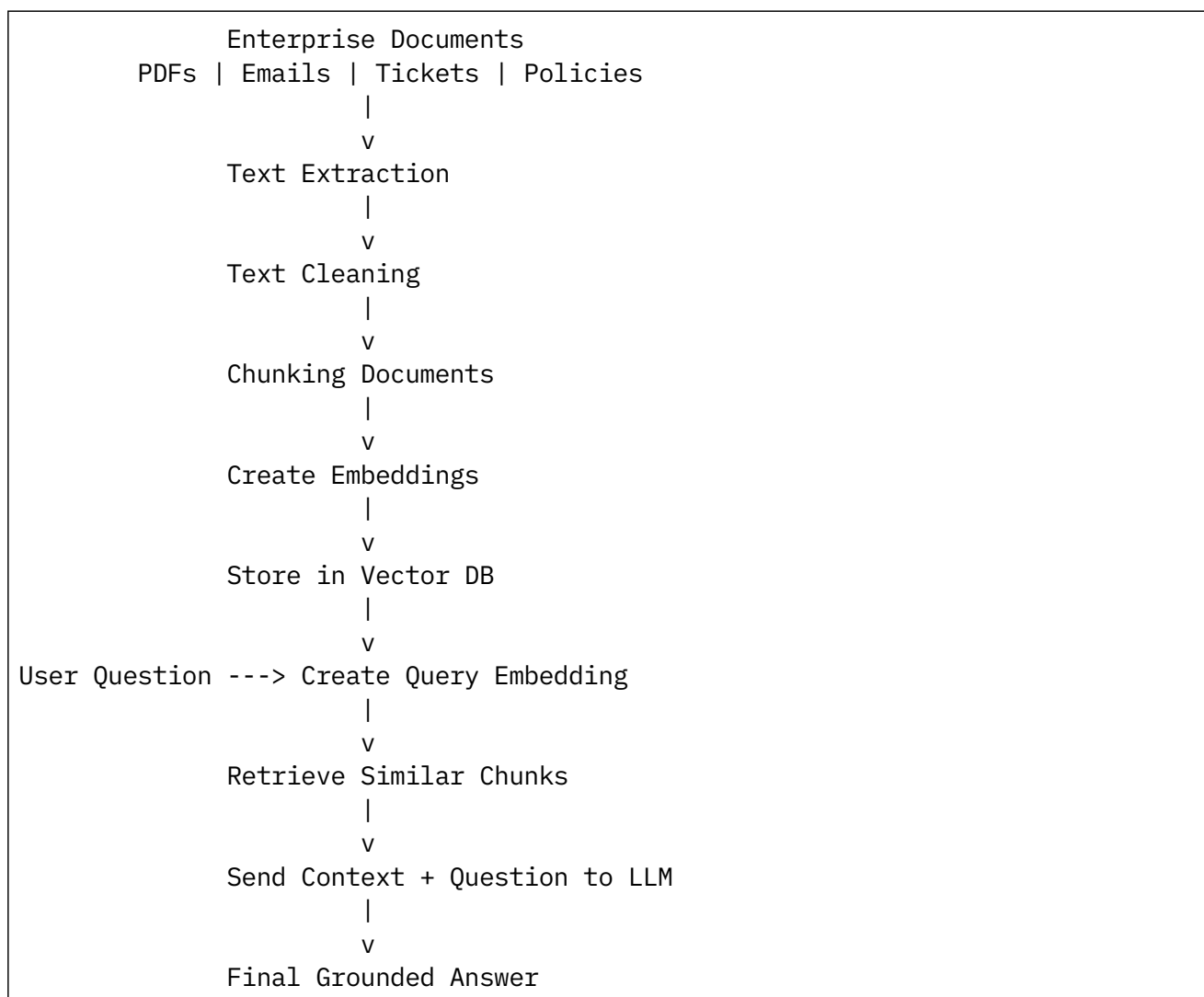
Document cleaning
Chunking
Embedding generation

Semantic search
Query understanding
Reranking
Answer generation
Entity extraction

16. ASCII diagram: NLP pipeline

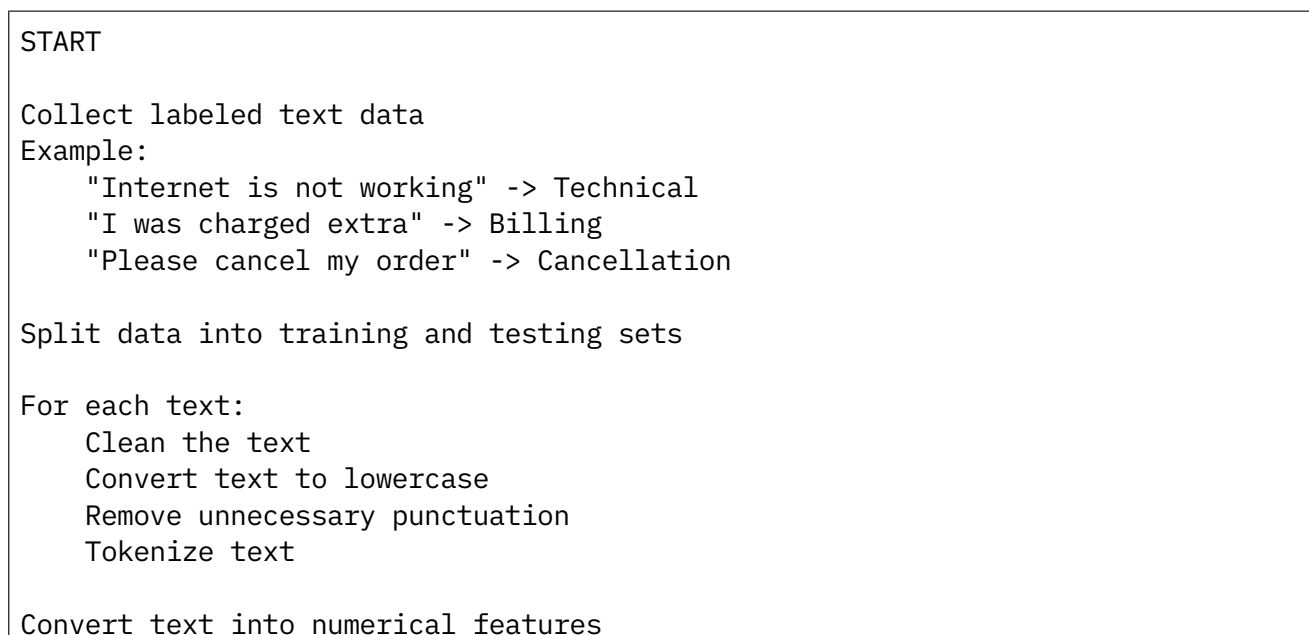


17. ASCII diagram: NLP connection to LLM and RAG



18. Pseudocode for text classification

Example use case: classify support tickets into categories.



Use TF-IDF or embeddings

Train classification model

Example: Logistic Regression or Naive Bayes

Evaluate model on test data

Check accuracy, precision, recall, F1-score

For new user text:

Clean the text

Convert text into same feature format

Send features to trained model

Predict category

Return predicted category

END

Python-style pseudocode

```
texts, labels = load_dataset()

cleaned_texts = []

for text in texts:
    text = lowercase(text)
    text = remove_noise(text)
    tokens = tokenize(text)
    cleaned_text = join(tokens)
    cleaned_texts.append(cleaned_text)

train_texts, test_texts, train_labels, test_labels = train_test_split(
    cleaned_texts,
    labels
)

vectorizer = create_tfidf_vectorizer()

train_vectors = vectorizer.fit_transform(train_texts)
test_vectors = vectorizer.transform(test_texts)

model = create_classifier()
model.train(train_vectors, train_labels)

predictions = model.predict(test_vectors)

evaluate_model(test_labels, predictions)

new_text = "My payment failed but money was deducted"
new_text_cleaned = preprocess(new_text)
new_vector = vectorizer.transform([new_text_cleaned])

predicted_category = model.predict(new_vector)
```

```
print(predicted_category)
```

19. Pseudocode for converting documents into embeddings

Example use case: prepare documents for RAG.

```
START

Collect documents
    PDFs
    Word files
    Web pages
    Support tickets
    Policy documents

Extract text from each document

For each document:
    Clean text
    Remove unnecessary spaces
    Remove headers/footers if needed

Split document into smaller chunks
    Example: 500 words per chunk
    Keep some overlap between chunks

For each chunk:
    Send chunk to embedding model
    Get embedding vector

Store in vector database:
    chunk text
    embedding vector
    document name
    page number
    metadata

When user asks a question:
    Convert question into embedding
    Search vector database for similar chunks
    Retrieve top matching chunks
    Send chunks + question to LLM
    Generate final answer

END
```

Python-style pseudocode

```
documents = load_documents(folder_path)

all_chunks = []
```

```

for document in documents:
    text = extract_text(document)
    text = clean_text(text)

    chunks = split_text(
        text,
        chunk_size=500,
        overlap=50
    )

    for chunk in chunks:
        embedding = embedding_model.encode(chunk)

        record = {
            "text": chunk,
            "embedding": embedding,
            "source": document.name,
            "page": document.page_number
        }

        all_chunks.append(record)

vector_database.insert(all_chunks)

user_question = "What is the refund policy?"

question_embedding = embedding_model.encode(user_question)

similar_chunks = vector_database.search(
    embedding=question_embedding,
    top_k=5
)

context = combine(similar_chunks)

prompt = create_prompt(
    context=context,
    question=user_question
)

answer = llm.generate(prompt)

print(answer)

```

20. Easy examples for each concept

Example 1: Text preprocessing

Raw:
 "Hello!!! I need refund for my FAILED payment."

Cleaned:
"hello need refund failed payment"

Example 2: Tokenization

Text:
"AI is useful"

Tokens:
["AI", "is", "useful"]

Example 3: Stop words

Text:
"The customer is asking for a refund"

After stop word removal:
"customer asking refund"

Example 4: Stemming

connected
connecting
connection

Stem:
connect

Example 5: Lemmatization

running -> run
better -> good
children -> child

Example 6: Bag of Words

Text 1:
"refund failed"

Text 2:
"payment failed"

Vocabulary:
["refund", "failed", "payment"]

Text 1 vector:
[1, 1, 0]

Text 2 vector:
[0, 1, 1]

Example 7: TF-IDF

Common word:
"the" appears in many documents, so low score.

Important word:
"hypothyroid" appears in fewer medical documents, so higher score.

Example 8: Word embedding

"doctor" and "hospital" will be close in vector space.

"doctor" and "banana" will be far apart.

Example 9: Sentence embedding

Sentence 1:
"I need my money back."

Sentence 2:
"Please process my refund."

These two sentences are semantically similar.

Example 10: Text classification

Input:
"My router is not working."

Output:
Technical Issue

Example 11: NER

Text:
"IBM opened a new office in Bengaluru."

Entities:
IBM -> Organization
Bengaluru -> Location

Example 12: Sentiment analysis

Text:
"The support team resolved my issue quickly."

Sentiment:
Positive

21. Traditional NLP vs modern GenAI NLP

Area	Traditional NLP	Modern GenAI / LLM
Feature creation	Manual, TF-IDF, BoW	Learned automatically
Input	Cleaned text often needed	Natural text works well
Model	Task-specific	General-purpose
Example model	Naive Bayes, SVM	Transformer, LLM
Output	Category, score, entity	Answer, summary, reasoning, extraction
Strength	Fast, cheap, explainable	Flexible, powerful
Weakness	Limited understanding	Cost, latency, hallucination risk

22. Where this matters in IBM AI/GenAI roles

For IBM AI/GenAI roles, NLP is useful in practical projects like:

Building enterprise chatbots
Creating RAG systems over internal documents
Classifying customer support tickets
Extracting entities from forms and contracts
Analyzing customer sentiment
Searching documents semantically
Summarizing business reports
Building watsonx-based GenAI solutions

In interviews, you may be asked:

How do you convert text into numbers?
What is the difference between TF-IDF and embeddings?
Why are embeddings important in RAG?
What is tokenization?
What is the difference between stemming and lemmatization?
How do you classify text?
How do you extract entities from documents?
How does NLP connect to LLM applications?

23. Common mistakes

Mistake 1: Thinking NLP only means chatbots

NLP is much broader.

It includes:

```
Search
Classification
NER
Sentiment analysis
Summarization
Translation
Document extraction
RAG
```

Mistake 2: Removing important stop words

For sentiment analysis, words like not, never, and no are very important.

Wrong:

```
"I am not happy" -> "happy"
```

This changes negative sentiment into positive sentiment.

Mistake 3: Using Bag of Words and expecting meaning

Bag of Words counts words but does not understand meaning.

```
"refund needed"
"money back required"
```

Bag of Words may not understand that both mean similar things.

Embeddings handle this better.

Mistake 4: Confusing word embeddings and sentence embeddings

Word embeddings represent single words.

Sentence embeddings represent full sentences, paragraphs, or chunks.

For RAG, sentence or document chunk embeddings are usually more useful.

Mistake 5: Using too much preprocessing with LLMs

For classical ML, cleaning is important.

For LLMs, excessive cleaning may remove useful context.

Example:

```
Original:
"The customer said: 'I am NOT satisfied!!!"

Over-cleaned:
"customer satisfied"
```

Meaning becomes wrong.

Mistake 6: Not preserving metadata in RAG

When storing embeddings, also store metadata.

Important metadata:

Document name
Page number
Section
Date
Source URL
Author
Department

Without metadata, it becomes hard to cite or verify answers.

Mistake 7: Poor chunking in RAG

If chunks are too small, meaning is lost.

If chunks are too large, retrieval becomes noisy.

Good chunking keeps enough context while staying focused.

24. One simple mental model

Think of NLP like this:

Human Language
v
Clean and break into pieces
v
Convert into numbers
v
Model understands patterns
v
Useful output

For modern GenAI:

Documents
v
Chunks + embeddings
v
Search relevant context

|
v
LLM generates answer

That is the foundation of NLP, LLMs, and RAG.